

1주 · 리눅스 & Docker

리눅스 커맨드 · 셸 스크립트 · Docker · Compose (5차시, 20H)

1-5 · Docker Compose 이해하고 사용하기

1-5 · Docker Compose 이해하고 사용하기

여러 컨테이너를 docker-compose.yml 하나로 정의해 통합 엔지니어링 환경을 완성하고, 주차 산출물로 제출합니다.

학습 목표

- Compose가 필요한 이유와 services·volumes·depends_on 구조를 설명할 수 있습니다.
- docker compose up·ps·logs·down으로 다중 컨테이너를 관리할 수 있습니다.
- 서비스 이름으로 컨테이너끼리 통신하는 것을 확인할 수 있습니다.
- db·admin·app 3개 서비스를 묶은 통합 환경과 README를 완성해 산출물로 제출합니다.

시간 배분 (4H)

구분	Chapter	시간
이론	Compose 개념과 YAML 구조	1H
실습	다중 컨테이너 구성 실습	1.5H
실습	주차 산출물: 통합 엔지니어링 환경 구축	1H
정리·회고	1주차 총정리·산출물 제출 확인	0.5H

참고 사전 준비: 4차시에 만든 ~/data-eng/week1/app 디렉터리 (Dockerfile·app.py·requirements.txt)가 그대로 필요합니다.

3차시의 pg 컨테이너가 아직 떠 있다면 docker stop pg로 정지해 5432 포트를 비워둡니다.

TIP 수업 시작 전에 3차 메뉴 맨 위 「실습점검 · 환경재설정」부터 따라 해 주세요.

Docker 실행 · 1-4의 app 폴더 · 5432·8080·8081 포트 · 이미지 미리 받기 · 오늘 바뀐 안내(pgAdmin 9.18)를 10분 안에 점검합니다.

실습 중에 꼬였을 때 처음부터 다시 하는 방법도 거기에 있습니다.

오늘 만들 것

실제 데이터 환경은 DB·관리도구·애플리케이션 여러 컨테이너가 한 팀으로 움직입니다.

이들을 docker run 명령 여러 개로 하나씩 띄우는 대신, 파일 하나에 적어 두고 한 번에 켜고 끄는 도구가 Docker Compose입니다.

1주차의 마지막 날입니다.

오늘 만드는 docker-compose.yml은 1주차 산출물이면서, 앞으로 7주 동안 계속 쓸 도구의 첫 사용이기도 합니다.

4주차 MinIO·Iceberg도, 5주차 Spark 클러스터도, 6주차 Kafka도 전부 이 형식의 파일 하나로 띄웁니다.

실습점검 · 환경재설정

수업을 시작하기 전 10분 — 1-4까지의 결과물과 포트·이미지를 점검하고, 오늘 바뀐 안내를 내 환경에 반영합니다.

1-5 · 수업 전 10분 · 실습점검 · 환경재설정 슬라이드 9장

슬라이드: <https://raw.githubusercontent.com/aeonlee/materials/main/data-eng/decks/w1d5s0.pdf>

이 메뉴는 무엇을 하나요

1-5는 1주차의 마지막 날이라, 1-3·1-4에서 만든 것 위에 오늘 실습이 올라갑니다.

그런데 앞 차시를 끝낸 모습이 사람마다 다릅니다 — 누구는 pg 컨테이너가 아직 돌고, 누구는 app 폴더가 없고, 누구는 이미지를 받아 두지 않았습니니다.

그래서 같은 명령을 쳐도 결과가 갈립니다.

이 메뉴는 그 차이를 수업 전에 없애는 곳입니다.

점검 (1~5단계)		재설정 (6단계)
언제	수업 시작 전에 한 번	실습이 꼬여서 처음부터 다시 하고 싶을 때만
무엇을	Docker · app 폴더 · 포트 · 이미지 · 오늘 바뀐 안내를 확인하고 모자란 것을 채운다	오늘 만든 Compose 환경만 내린다
지우는 것	없음	compose-test와 week1 환경 · week1_pgdata 볼륨(down -v)
건드리지 않는 것	—	1-4의 app 폴더 · 3차시의 pgdata 볼륨

재설정은 되돌아갈 지점을 정해 두는 일입니다.

실습 중에 설정이 꼬이면 원인을 한참 찾기보다 6단계로 오늘 만든 것만 내리고 다중 컨테이너 실습 1단계부터 다시 하는 편이 빠릅니다.

앞 차시 결과물은 남겨 두므로 1주차 전체를 다시 할 필요는 없습니다.

참고 운영체제는 Windows(WSL2 우분투) · macOS · 리눅스(Ubuntu) 세 갈래입니다. 1단계(Docker 켜기)와 3단계(포트 확인)만 운영체제별 탭으로 나뉘고, 나머지는 셋 모두 같은 명령입니다.

Windows는 모든 명령을 우분투 터미널에서 칩니다 — 3단계의 포트 확인 한 줄만 PowerShell입니다.

1 Docker가 켜져 있는지 확인합니다

Windows (WSL2)

Docker Desktop을 실행하고 작업 표시줄의 고래 아이콘이 멈출 때까지 기다립니다.

Desktop의 Settings → Resources → WSL Integration에서 Ubuntu가 켜져 있어야 우분투 터미널에서 docker를 쓸 수 있습니다.

명령은 우분투 터미널에서 칩니다.

```
bash
```

```
1 docker version --format '{{.Server.Version}}'
```

우분투에서 docker: command not found가 나오면 WSL Integration이 꺼져 있는 것입니다.
컨 뒤 우분투 터미널을 새로 엽니다.

macOS

Docker Desktop을 실행하고 메뉴 막대의 고래 아이콘이 멈출 때까지 기다린 뒤 터미널에서
칩니다.

```
bash
```

```
1 docker version --format '{{.Server.Version}}'
```

리눅스 (Ubuntu)

리눅스는 Desktop 없이 Docker 엔진(데몬)이 서비스로 돕니다.
꺼져 있으면 켜고, 재부팅 뒤에도 켜지게 해 돕니다.

```
bash
```

```
1 sudo systemctl enable --now docker
2 docker version --format '{{.Server.Version}}'
```

permission denied ... docker.sock이 나오면 내 계정이 docker 그룹에 없는 것입니다.
sudo usermod -aG docker \$USER 뒤 newgrp docker(지금 터미널만) 또는 로그아웃·
로그인으로 적용합니다.

확인 포인트: 29.6.1처럼 버전 숫자 한 줄이 나오면 됩니다.

Cannot connect to the Docker daemon은 명령이 틀린 것이 아니라 Docker(일할 쪽)가 꺼져
있다는 뜻입니다 — 위 탭대로 켜고 다시 칩니다.

실제 실행 화면(macOS) — Docker가 켜져 있으면 서버 버전 한 줄(29.6.1)만 나옵니다. 꺼져 있으면 이 자리에 Cannot connect to the Docker daemon이 나옵니다.

2 1-4의 app 폴더가 있는지 확인합니다

bash

```
1 ls ~/data-eng/week1/app
```

확인 포인트: Dockerfile · app.py · requirements.txt 세 파일이 보이면 됩니다.

Dockerfile.bad·Dockerfile.fat·Dockerfile.multi가 함께 보이는 것도 정상입니다(1-4 실험 파일).

실제 실행 화면 — app.py · Dockerfile · requirements.txt 세 파일과 1-4 실험 파일 (Dockerfile.bad·fat·multi)이 함께 보입니다.

폴더가 없거나 파일이 빠졌다면 아래를 그대로 붙여 넣어 최소 구성을 만듭니다.

1-4 1~2단계와 같은 내용입니다.

bash

```

1  mkdir -p ~/data-eng/week1/app
2  cd ~/data-eng/week1/app
3
4  cat > app.py << 'EOF'
5  import platform
6
7  print("데이터 파이프라인 앱이 실행되었습니다.")
8  print("파이썬 버전:", platform.python_version())
9  EOF
10
11 cat > requirements.txt << 'EOF'
12 requests
13 EOF
14
15 cat > Dockerfile << 'EOF'
16 FROM python:3.12-slim
17 WORKDIR /app
18 COPY requirements.txt .
19 RUN pip install --no-cache-dir -r requirements.txt
20 COPY . .
21 CMD ["python", "app.py"]
22 EOF
23
24 ls

```

3 5432·8080·8081 포트를 비웁니다

오늘 쓰는 포트는 세 개입니다.

5432는 PostgreSQL, 8080은 연습용 nginx, 8081은 pgAdmin입니다.

1-3에서 띄운 pg·web·web2가 아직 돌고 있으면 오늘 컨테이너가 뜨지 못합니다(port is already allocated).

bash

```

1  # 지금 돌고 있는 컨테이너와 포트
2  docker ps --format 'table {{.Names}}\t{{.Ports}}'
3
4  # 1-3에서 쓴 이름들을 멈춥니다 - 없는 이름은 오류만 내고 넘어갑니다
5  docker stop pg web web2 2>/dev/null

```

도커가 아닌 프로그램이 포트를 잡고 있을 수도 있습니다.

컴퓨터에 PostgreSQL을 직접 설치해 둔 경우가 대표적입니다.

운영체제에 맞는 탭으로 확인합니다.

Windows (WSL2)

powershell

```
1 netstat -ano | findstr ":5432 :8080 :8081"
```

이 한 줄만 PowerShell에서 칩니다.

Windows에 따로 설치한 프로그램(PostgreSQL 등)은 우분투 터미널에서는 보이지 않기 때문입니다.

LISTENING 줄이 없으면 비어 있는 것입니다.

있으면 맨 끝 숫자(PID)를 작업 관리자 → 세부 정보 탭에서 찾아 어떤 프로그램인지 봅니다.

Windows에 PostgreSQL을 따로 설치했다면 서비스(services.msc)에서 postgresql 서비스를 중지합니다.

macOS

bash

```
1 lsof -nP -iTCP -sTCP:LISTEN | grep -E ':(5432|8080|8081) '
```

아무것도 안 나오면 세 포트가 모두 비어 있는 것입니다.

맨 앞 이름이 com.docker로 시작하면 도커 컨테이너가 잡고 있는 것이니 위의 docker ps로 이름을 찾아 멈춥니다.

postgres처럼 다른 이름이면 그 프로그램을 끕니다 — Homebrew로 설치했다면 brew services list로 이름을 보고 brew services stop 이름.

리눅스 (Ubuntu)

bash

```
1 sudo ss -ltnp | grep -E ':(5432|8080|8081) '
```

아무것도 안 나오면 비어 있는 것입니다.

맨 끝 users:(("...""))에 docker-proxy가 보이면 도커 컨테이너가 잡고 있는 것이니 위의 docker

ps로 찾아 멈춥니다.

postgres가 보이면 apt로 설치한 PostgreSQL이 도는 것이니 `sudo systemctl stop postgresql`로 멈춥니다.

확인 포인트: docker ps에 pg-web-web2가 없고, 위 확인 명령에 LISTEN(ING) 줄이 나오지 않으면 됩니다.

실제 실행 화면(macOS) — docker ps가 제목 줄(NAMES PORTS)만 내면 돌고 있는 컨테이너가 없는 것입니다. docker stop은 이미 멈춰 있던 pg-web2의 이름을 그대로 찍고, 없는 이름 web의 오류는 2>/dev/null로 숨겨져 조용히 넘어갑니다. lsof가 아무 줄도 내지 않으면 세 포트가 비어 있는 것입니다 — 그 뒤 프롬프트 화살표가 빨간 것은 grep이 찾은 줄이 없다는 표시라 정상입니다.

4 오늘 쓸 이미지를 미리 받아 둡니다

실습 중에 이미지를 받기 시작하면 그동안 손을 놓고 기다려야 합니다.

pgAdmin 이미지가 디스크에서 약 790MB라 가장 오래 걸립니다(macOS 실측 31.8~43.6초 — 여럿이 동시에 받으면 더 걸립니다).

이론 시간이 시작될 때 걸어 두면 실습 전에 끝납니다.

bash

```
1 # docker pull은 한 번에 이미지 하나만 받으므로 for로 돌립니다
2 for i in dpape/pgadmin4:9.18 postgres:16 nginx:1.27
  curlimages/curl:8.10.1; do docker pull $i; done
3
4 # 받은 것 확인
5 docker images --format '{{.Repository}}:{{.Tag}}' | grep -E
  'pgadmin4|postgres|nginx|curl'
```

확인 포인트: 마지막 명령에 dpape/pgadmin4:9.18 · postgres:16 · nginx:1.27 · curlimages/curl:8.10.1 네 줄이 모두 보이면 됩니다.

실제 실행 화면 — 이미 받아 둔 이미지는 Image is up to date로 바로 넘어갑니다. 처음 받는 분은 이 자리에 내려받는 진행 표시가 한동안 나옵니다. 사이사이의 What's next 안내는 무시해도 됩니다. 마지막 명령에 네 줄이 보이면 됩니다.

5 오늘 바뀐 안내를 반영합니다

pgAdmin 이미지 이름이 dpage/pgadmin4에서 dpage/pgadmin4:9.18로 바뀌었습니다. 버전을 적어 두면 누가 언제 받아도 같은 이미지가 됩니다(postgres:16·nginx:1.27처럼). 지금 받아지는 것과 같은 이미지라 동작은 달라지지 않습니다.

이미 ~/data-eng/week1/docker-compose.yml을 만들어 둔 분만 아래를 칩니다. 아직 안 만들었다면 건너뛰니다 — 3단계 본문에 이미 반영되어 있습니다.

bash

```
1 cd ~/data-eng/week1
2
3 # 태그 없는 줄만 9.18로 바꿉니다 — 여러 번 쳐도 9.18:9.18처럼 겹치지 않습니다
4 sed -i.bak 's|image: dpage/pgadmin4$|image: dpage/pgadmin4:9.18|'
   docker-compose.yml
5 grep pgadmin docker-compose.yml
```

확인 포인트: grep 결과가 image: dpage/pgadmin4:9.18 한 줄이면 됩니다.
 옆에 생긴 docker-compose.yml.bak은 바꾸기 전 원본이니 지워도 됩니다.
 sed -i.bak은 macOS·리눅스·WSL 어디서나 같게 동작하는 형태입니다.

실제 실행 화면 — 이미 9.18이 적혀 있던 파일이라 sed가 바꿀 것이 없었고, grep 결과는 image: dpage/pgadmin4:9.18 한 줄입니다. 태그가 없던 파일이라도 결과는 같은 한 줄이 됩니다.

3차시의 pgdata 볼륨은 오늘 쓰지 않습니다.

Compose는 week1_pgdata라는 새 볼륨을 따로 만듭니다 — 이유는 다중 컨테이너 실습 3단계의 「볼륨 이름 체크」에 있습니다.

pgdata는 지우지 않고 그대로 둡니다.

6 처음부터 다시 하고 싶을 때 (환경 재설정)

실습 중에 꼬여서 처음부터 다시 하고 싶으면 오늘 만든 것만 깨끗이 내립니다.
 앞 차시의 결과물(app 폴더·pgdata)은 건드리지 않습니다.

bash

```
1 # 지금 떠 있는 Compose 묶음 보기
2 docker compose ls
3
4 # 1단계 연습용(compose-test) 내리기
5 cd ~/data-eng/week1/compose-test && docker compose down
6
7 # 통합 환경 내리기 - -v 는 week1_pgdata 볼륨까지 지웁니다(되돌릴 수 없음)
8 cd ~/data-eng/week1 && docker compose down -v
9
10 # 확인
11 docker compose ls
```

확인 포인트: 마지막 docker compose ls에 제목 줄(NAME STATUS CONFIG FILES)만 나오면 재설정이 끝난 것입니다.

이제 다중 컨테이너 실습 1단계부터 다시 시작하면 됩니다.

주의 down -v는 week1 표처럼 오늘 DB에 넣은 데이터를 모두 지웁니다.
주차 산출물 1단계(데이터 보존 확인)를 이미 마쳤다면 -v 없이 docker compose
down만 칩니다.
pgAdmin에 등록한 서버도 컨테이너를 새로 만들면 사라지니 다시 등록합니다.

Compose 개념과 YAML 구조 (1H)

YAML 문법과 Compose 파일의 뼈대를 잡습니다. 서비스 이름으로 서로를 부르는 규칙이 오늘의 핵심입니다.

1-5 · 이론 1H · Compose 개념과 YAML 구조 슬라이드 10장

슬라이드: <https://raw.githubusercontent.com/aeonlee/materials/main/data-eng/decks/w1d5s1.pdf>

Compose가 푸는 문제

1-4까지는 컨테이너 하나를 docker run으로 띄웠습니다.

그런데 실무 환경은 최소 서너 개입니다.

DB 하나, 관리 도구 하나, 내 앱 하나.

이걸 매번 docker run 세 번으로 띄우면 옵션을 기억해야 하고, 순서를 지켜야 하고, 정리할 때도 하나씩 지워야 합니다.

- Compose = 오케스트라 지휘자: 연주자(컨테이너)마다 따로 신호를 주는 대신, 악보(docker-compose.yml) 하나를 보고 지휘자가 전체를 한 번에 시작·정지시킵니다.
- 무엇보다 파일로 남습니다. Git에 올려 두면 다른 사람이 같은 환경을 한 줄로 재현합니다.
- "제 컴퓨터에서는 되는데요"를 없애는 마지막 조각이 이것입니다. 컨테이너가 앱 하나의 환경을 고정했다면, Compose는 환경 전체를 고정합니다.

YAML 문법 — 들여쓰기가 곧 구조입니다

YAML은 설정을 적는 형식입니다.

앞으로 Compose 뿐 아니라 Kafka·Spark 설정에서도 계속 만납니다.

규칙은 몇 개 안 되지만, 어기면 파일이 통째로 안 읽힙니다.

yaml

```

1 # 1. 키: 값 (콜론 뒤에 공백 하나가 필요합니다)
2 name: postgres
3
4 # 2. 들여쓰기로 계층을 만듭니다 - 공백 2칸이 관례, 탭은 금지
5 database:
6   host: db
7   port: 5432
8
9 # 3. 목록은 하이픈(-)
10 ports:
11   - "5432:5432"
12   - "8081:80"
13
14 # 4. 주석은 #
15 # 5. 숫자로 시작하거나 콜론이 든 값은 따옴표로 감쌉니다
16 value: "8081:80"      # 따옴표가 없으면 60진수 숫자(시:분처럼)로 읽힐 수 있습니다

```

주의 탭 문자는 YAML에서 금지입니다.

편집기가 자동으로 탭을 넣는 설정이면 파일이 안 읽힙니다.

VS Code 라면 우측 하단에서 "Spaces: 2"로 되어 있는지 확인하세요.

오늘 가장 많이 만나실 오류입니다.

Compose 파일의 뼈대

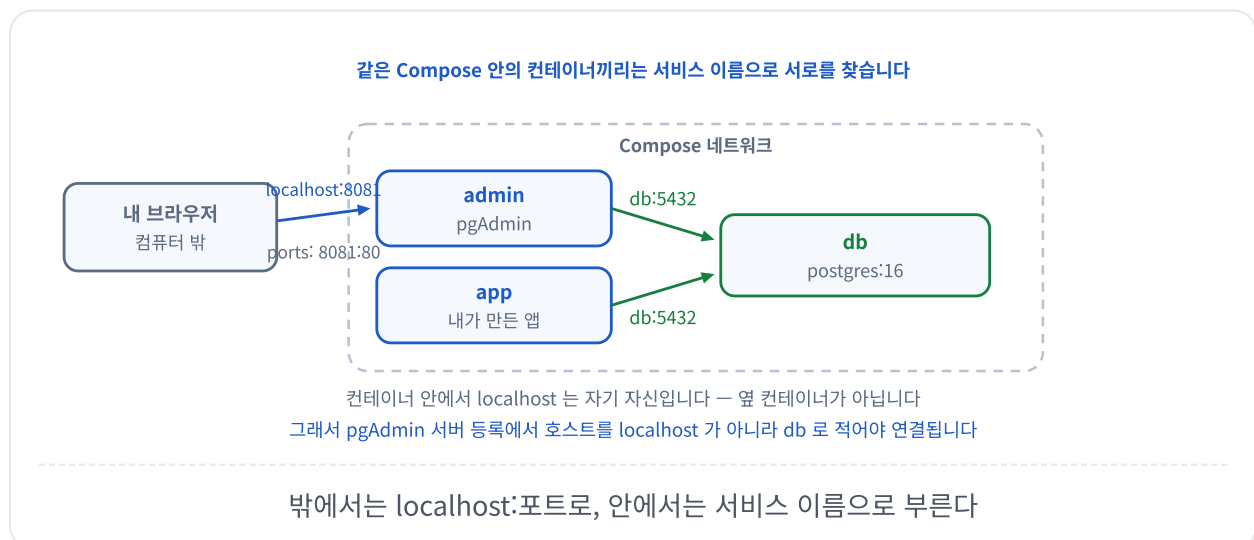
최상위 키	무엇을 적나
services	띄울 컨테이너들의 목록. 여기가 본체입니다
volumes	서비스들이 쓰는 이름 볼륨 선언
networks	네트워크를 나누고 싶을 때 (안 적으면 자동으로 하나 만들어 줍니다)

서비스 안의 키	무엇을 적나	docker run으로 치면
image	가져다 쓸 이미지	마지막 인자
build	직접 빌드할 디렉터리 (Dockerfile 위치)	docker build
ports	포트 매핑 목록	-p
volumes	볼륨·바인드 마운트 목록	-v
environment	환경변수	-e
depends_on	먼저 시작할 서비스	(없음 — 순서를 손으로 지켜야 했음)
command	기본 명령 덮어쓰기	run 뒤에 붙이던 명령

서비스 이름이 곧 주소가 됩니다

오늘 가장 중요한 규칙입니다.

Compose는 서비스들을 같은 네트워크에 넣고, 서비스 이름을 호스트 이름으로 쓸 수 있게 해 줍니다. 같은 반 친구를 주소가 아니라 이름으로 부르는 것과 같습니다.



- 컨테이너 안에서 다른 컨테이너를 부를 때는 서비스 이름을 씁니다 — db:5432
- 내 컴퓨터(브라우저·터미널)에서 부를 때는 localhost:매핑한포트를 씁니다 — localhost:8081
- 컨테이너 안의 localhost는 그 컨테이너 자기 자신입니다. 옆 컨테이너가 아닙니다.

- 그래서 pgAdmin에서 서버를 등록할 때 호스트를 localhost로 적으면 연결에 실패합니다. db 라고 적어야 합니다.

depends_on의 함정

depends_on은 "시작 순서"만 정합니다.

db가 먼저 시작되기는 하지만, db가 접속을 받을 준비를 마쳤다는 뜻은 아닙니다.

PostgreSQL은 프로세스가 뜬 뒤에도 초기화에 몇 초가 더 걸립니다.

yaml

```

1  # 이렇게만 쓰면 app이 db보다 먼저 접속을 시도할 수 있습니다
2  app:
3    depends_on:
4      - db
5
6  # 준비 완료까지 기다리게 하려면 healthcheck와 함께 씁니다
7  db:
8    image: postgres:16
9    healthcheck:
10      test: ["CMD-SHELL", "pg_isready -U engineer"]
11      interval: 5s
12      retries: 5
13
14  app:
15    depends_on:
16      db:
17        condition: service_healthy

```

TIP 실무에서는 healthcheck를 쓰더라도 앱 쪽에 재시도 로직을 함께 둡니다.
네트워크는 언제든 끊길 수 있고, DB는 운영 중에도 잠깐 못 받을 수 있기 때문입니다.
오늘은 개념만 알아 두고, 심화 과제에서 직접 적용해 봅니다.

이 1시간에서 꼭 가져갈 것

- YAML은 들여쓰기가 구조다. 공백 2칸, 탭 금지.
- services 아래에 컨테이너를 하나씩 적고, image 또는 build로 무엇을 띄울지 정한다.
- 컨테이너끼리는 서비스 이름으로, 밖에서는 localhost:포트로 부른다.
- depends_on은 시작 순서일 뿐 준비 완료가 아니다.
- docker run 옵션과 Compose 키는 일대일로 대응한다.

다중 컨테이너 구성 실습 (1.5H)

두 개짜리 작은 Compose부터 시작해 서비스 간 통신을 확인하고, 세 개짜리 통합 환경으로 늘립니다.

1-5 · 실습 1.5H · 다중 컨테이너 구성 실습 슬라이드 11장

슬라이드: <https://raw.githubusercontent.com/aebonlee/materials/main/data-eng/decks/w1d5s2.pdf>

1 가장 작은 Compose 파일부터

먼저 서비스 하나짜리로 시작해 up·ps·logs·down 한 바퀴를 돌아 봅니다.

bash

```
1 mkdir -p ~/data-eng/week1/compose-test
2 cd ~/data-eng/week1/compose-test
3
4 cat > docker-compose.yml << 'EOF'
5 services:
6   web:
7     image: nginx:1.27
8     ports:
9       - "8080:80"
10 EOF
11
12 docker compose up -d
13 docker compose ps
14 curl http://localhost:8080
15 docker compose logs web
16 docker compose down
17 docker compose ps -a      # 제목 줄만 나오면 정리 끝
```

확인 포인트: docker compose ps에 web이 running으로 보이고, curl에 "Welcome to nginx!"가 나오고, down 후 docker compose ps -a에 제목 줄(NAME IMAGE ...)만 나오고 그 아래가 비어 있으면 성공입니다.

docker ps -a는 3차시의 pg처럼 이 폴더와 상관없는 컨테이너까지 보여 주므로, 이 폴더의 Compose가 만든 것만 보는 docker compose ps -a로 확인합니다.

① `up -d`로 web 하나가 뜨고(Started 0.4초), `compose ps`의 STATUS가 Up, `curl`에는 「Welcome to nginx!」 HTML이 돌아옵니다.

② `logs web`에는 nginx 기동 기록과 방금 `curl`이 남긴 요청 한 줄('GET / HTTP/1.1' 200)이 있습니다. `down` 하면 컨테이너와 네트워크가 함께 Removed 되고, 마지막 `docker compose ps -a`에는 제목 줄만 남습니다.

TIP `docker compose` 명령은 `docker-compose.yml`이 있는 폴더에서 실행해야 합니다.

다른 곳에서 치면 "no configuration file provided"가 납니다.

2 서비스를 둘로 늘려 이름으로 통신해 봅니다

bash

```

1 cat > docker-compose.yml << 'EOF'
2 services:
3   web:
4     image: nginx:1.27
5     ports:
6       - "8080:80"
7
8   client:
9     image: curlimages/curl:8.10.1
10    command: sleep infinity # 계속 떠 있게 해서 실험에 씁니다
11    depends_on:
12      - web
13 EOF
14
15 docker compose up -d
16
17 # client 안에서 web을 서비스 이름으로 불러 봅니다
18 docker compose exec client curl -s http://web:80 | head -n 5
19
20 # 같은 자리에서 localhost로 부르면 실패합니다 (자기 자신을 가리키므로)
21 docker compose exec client curl -s --max-time 3
    http://localhost:80 || echo "localhost 로는 실패 - 정상입니다"

```

주의 `docker compose exec` 명령은 **한 줄씩** 붙여 넣고, 결과가 나온 것을 본 뒤 다음 줄로 넘어갑니다.

여러 줄을 한꺼번에 붙여 넣으면 뒤 줄이 터미널에서 실행되지 않고, 앞의 `exec` 가 연 컨테이너의 입력(stdin)으로 빨려 들어갑니다.

그러면 뒤 줄은 결과 없이 사라져 「쳤는데 아무 일도 없다」 처럼 보입니다.

오늘 `exec` 가 나오는 곳(2단계·5단계·산출물 1단계)은 모두 이렇게 한 줄씩 칩니다.

확인 포인트: `http://web:80`은 nginx HTML을 돌려주고, `http://localhost:80`은 실패해야 성공입니다.

이론에서 본 "컨테이너 안의 localhost는 자기 자신"을 직접 확인한 것입니다.

실제 실행 화면 — client 안에서 http://web:80은 nginx HTML을 돌려주고, http://localhost:80은 실패해 「localhost 로는 실패 — 정상입니다」가 찍힙니다. client의 localhost는 client 자신입니다.

bash

- 1 # 정리하고 상위 폴더로
- 2 **docker** compose down
- 3 **cd** ~/data-eng/week1

3 통합 환경의 docker-compose.yml을 작성합니다

이제 본 실습입니다.

~/data-eng/week1/docker-compose.yml로 저장합니다.

4차시에 만든 app 폴더가 같은 위치에 있어야 build: ./app이 동작합니다.

yaml

```
1 services:
2   db:
3     image: postgres:16
4     environment:
5       POSTGRES_USER: engineer
6       POSTGRES_PASSWORD: engineer123
7       POSTGRES_DB: pipeline
8     ports:
9       - "5432:5432"
10    volumes:
11      - pgdata:/var/lib/postgresql/data
12
13   admin:
14     image: dpage/pgadmin4:9.18
15     environment:
16       PGADMIN_DEFAULT_EMAIL: admin@example.com
17       PGADMIN_DEFAULT_PASSWORD: admin123
18     ports:
19       - "8081:80"
20     depends_on:
21       - db
22
23   app:
24     build: ./app
25     depends_on:
26       - db
27
28 volumes:
29   pgdata:
```

bash

```
1 cd ~/data-eng/week1
2
3 # 파일이 제대로 만들어졌는지 문법 검사 ( 실행하지 않고 해석만 합니다)
4 docker compose config
5
6 # app 폴더가 옆에 있는지 확인
7 ls
8 ls app
```

확인 포인트: docker compose config가 오류 없이 해석 결과를 출력하면 YAML 문법이 맞는 것입니다.

그리고 ls app에 Dockerfile이 보여야 합니다.

실제 실행 화면 — 짧게 쓴 depends_on: - db가 condition: service_started로 풀려 있습니다.
 「db가 시작만 되면」 넘어간다는 뜻입니다(이론의 depends_on 함정). 네트워크 week1_default와 볼륨 week1_pgdata는 폴더 이름 week1을 앞에 붙여 만들어집니다. 맨 아래 ls app에 Dockerfile이 있습니다.

TIP docker compose config는 up 하기 전에 문법을 확인하는 좋은 습관입니다.
 오타 하나 때문에 컨테이너를 띄웠다 지웠다 하는 시간을 아껴 줍니다.

Aebon's Tip!

볼륨 이름 체크

- ✓ Compose는 볼륨·네트워크 이름 앞에 폴더 이름을 붙입니다 — 여기 적은 pgdata는 실제로 week1_pgdata, 네트워크는 week1_default
 - ✓ 그래서 3차시에 docker run -v pgdata로 만든 pgdata와는 다른 볼륨입니다 — docker volume ls에 둘 다 보입니다
 - ✓ 3차시 데이터를 이어 쓰지 않는 이유 — 3차시 pg는 POSTGRES_USER 없이 만들어 계정이 postgres입니다.
- postgres 이미지는 볼륨이 비어 있을 때만 계정·DB를 만들기 때문에, 채워진 볼륨을 붙이면 오늘 적은 engineer 계정과 pipeline DB가 생기지 않아 접속이 실패합니다

✓ 3차시의 pgdata는 그대로 두어도 됩니다 — docker volume rm은 되돌릴 수 없습니다

4 전체 환경을 한 번에 기동합니다

bash

```
1 # 3차시의 pg 컨테이너가 5432를 쓰고 있으면 먼저 정지
2 docker stop pg 2>/dev/null
3
4 # 빌드 + 기동을 한 번에
5 docker compose up -d --build
6
7 # 서비스 상태 확인
8 docker compose ps -a
9
10 # app 서비스 출력 확인
11 docker compose logs app
```

확인 포인트: docker compose ps -a에 db와 admin이 running으로 보이고, logs app에 4차시 앱의 출력("데이터 파이프라인 앱이 실행되었습니다.")이 남아 있으면 성공입니다. app은 출력 후 종료되는 일회성 앱이므로 exited (0) 상태가 정상입니다.

실제 실행 화면 — pgAdmin 이미지는 처음이라 내려받는 데 31.8초가 걸렸습니다(이미지 이름 dpage/pgadmin4:9.18 — 버전을 고정해 두어 누가 언제 받아도 같은 이미지입니다). app은 4차시 캐시로 빌드되고(RUN pip install CACHED), ps -a에 admin · db는 Up, app은 Exited (0)입니다. logs app에 4차시 앱의 출력이 남아 있습니다.

5 서비스 간 연결을 확인합니다

bash

```
1 # db 서비스의 psql로 직접 확인
2 docker compose exec db psql -U engineer -d pipeline -c "SELECT
  current_database();"
3
4 # admin 컨테이너 안에서 db 이름으로 접속되는지 확인
5 docker compose exec admin sh -c "getent hosts db"
```

확인 포인트: psql 결과에 pipeline이 출력되고, getent hosts db가 IP 주소와 db를 함께 보여 주면 이름 해석이 되고 있는 것입니다.

실제 실행 화면 — psql이 current_database = pipeline을 돌려주고, admin 컨테이너 안에서 db라는 이름이 172.18.0.2로 풀립니다.

브라우저에서 <http://localhost:8081>로 pgAdmin에 접속합니다(admin@example.com / admin123).

서버를 새로 등록할 때 Connection 탭의 Host name/address에 반드시 db를 입력하세요. localhost를 넣으면 연결되지 않습니다.

확인 포인트: pgAdmin에서 호스트 db · 사용자 engineer · 비밀번호 engineer123 · DB pipeline으로 서버가 연결되고 왼쪽 트리에 pipeline 데이터베이스가 보이면 성공입니다.

① 서버 등록 → Connection 탭 — Host name/address에 db, Maintenance database에 pipeline, Username에 engineer를 넣습니다. localhost가 아니라 db입니다.

② 저장하면 왼쪽 트리에 week1-db → Databases (2) → pipeline이 보입니다. postgres는 PostgreSQL이 기본으로 만드는 DB입니다. 오른쪽 그래프가 비어 있는 것은 고장이 아닙니다. 대시보드는 몇 초마다 세션 수·트랜잭션·읽고 쓴 행 수를 점으로 찍어 선을 이어 가는데, pipeline을 누른 직후라 찍힌 점이 없고 그 순간 DB에 들어온 쿼리도 없었기 때문입니다.

③ DB를 움직이면 그래프가 살아납니다. 이 화면은 그래프가 움직이는 모습을 보여 드리려고 캡처 때 따로 부하를 준 것입니다 — 표에 행을 넣고 개수를 세는 쿼리를 1초마다 반복하며 1분쯤 지켜봤습니다. 실습에서는 따라 하지 않습니다. 여기서 week1 표를 미리 만들면 주차 산출물 1단계의 SELECT 결과가 「1주차 완료」 한 줄이 아니게 됩니다. 쿼리를 시작한 시점부터 Transactions per second(초당 트랜잭션)·Tuples in(넣은 행)·Tuples out(읽은 행)·Block I/O가 함께 올라가고, 쿼리를 멈추자 선이 다시 0으로 내려왔습니다. 그래프는 지금 DB에 들어오는 일을 그대로 따라 그립니다.

주차 산출물: 통합 엔지니어링 환경 구축 (1H)

데이터 보존을 확인하고 README를 붙여 1주차 산출물을 완성해 제출합니다.

1-5 · 실습 1H · 주차 산출물: 통합 엔지니어링 환경 구축 슬라이드 9장

슬라이드: <https://raw.githubusercontent.com/aebonlee/materials/main/data-eng/decks/w1d5s3.pdf>

참고 이번 챕터의 결과물이 1주차 산출물입니다.

docker-compose.yml과 README.md 두 파일을 제출합니다.

제출은 사이트의 "산출물 제출" 페이지에서 하고, 제출 후 주차별 산출물 갤러리에도 올려 동료와 공유해 주세요.

1 데이터 보존을 확인합니다

bash

```
1 cd ~/data-eng/week1
2
3 # 테스트 표 생성
4 docker compose exec db psql -U engineer -d pipeline -c "CREATE
  TABLE IF NOT EXISTS week1 (memo TEXT);"
5 docker compose exec db psql -U engineer -d pipeline -c "INSERT
  INTO week1 VALUES ('1주차 완료');"
6
7 # 환경 전체 종료 후 재기동 (볼륨은 유지됩니다)
8 docker compose down
9 docker compose up -d
10
11 # 표와 데이터가 살아 있는지 확인
12 docker compose exec db psql -U engineer -d pipeline -c '\dt'
13 docker compose exec db psql -U engineer -d pipeline -c "SELECT *
  FROM week1;"
```

확인 포인트: down 후 up을 했는데도 week1 표와 "1주차 완료" 행이 남아 있으면, 볼륨이 데이터를 보존한다는 것을 확인한 것입니다.

실제 실행 화면 — down 뒤 다시 up 했는데도 표 목록에 week1이, SELECT에 「1주차 완료」 행이 그대로 있습니다. 볼륨 week1_pgdata가 데이터를 지켰습니다.

주의 docker compose down -v는 볼륨까지 지웁니다.
-v를 붙이면 방금 확인한 데이터가 사라지니 주의하세요.
평소에는 -v 없이 down만 씁니다.

2 README.md를 작성합니다

산출물의 핵심은 "다른 사람이 이 파일만 보고 같은 환경을 재현할 수 있는가"입니다.
아래를 뼈대로 쓰되, 본인이 겪은 문제와 해결을 한 줄이라도 추가해 주세요.

markdown

```

1 # 1주차 산출물 – Docker Compose 기반 통합 엔지니어링 환경
2
3 ## 구성
4 - db: PostgreSQL 16 (포트 5432, 볼륨 pgdata로 데이터 보존)
5 - admin: pgAdmin 4 (http://localhost:8081)
6 - app: 자체 빌드 파이썬 앱 (./app의 Dockerfile)
7
8 ## 실행 방법
9 1. docker compose up -d --build
10 2. docker compose ps로 상태 확인
11 3. 종료는 docker compose down
12
13 ## 접속 정보
14 - pgAdmin: admin@example.com / admin123
15 - DB: 호스트 db(내부) 또는 localhost:5432(외부), engineer /
    engineer123, DB명 pipeline
16
17 ## 확인한 것
18 - pgAdmin에서 호스트 이름을 db로 지정해 연결 성공
19 - docker compose down 후 up 해도 week1 표가 유지됨 (볼륨 동작 확인)
20
21 ## 겪은 문제와 해결
22 - (여기에 본인이 막혔던 지점과 해결 방법을 적어 주세요)

```

bash

```

1 # 위 내용을 파일로 저장한 뒤 확인
2 ls ~/data-eng/week1
3 cat ~/data-eng/week1/README.md

```

확인 포인트: README만 보고 다른 사람이 docker compose up -d --build부터 pgAdmin 접속까지 재현할 수 있으면 산출물 완성입니다.

실제 실행 화면 — README.md 예시입니다. 마지막 「겪은 문제와 해결」에는 이 실습에서 실제로 만난 load metadata 오류를 적었습니다.

3 제출 전 스스로 검증합니다

제출하기 전에 "처음 받는 사람"의 입장으로 한 번 돌려 보세요.
환경을 완전히 내리고 처음부터 올리는 것이 가장 확실한 검증입니다.

bash

```
1 cd ~/data-eng/week1
2
3 # 완전히 내리기 (이번에는 볼륨까지 - 처음부터 올라오는지 보기 위해)
4 docker compose down -v
5
6 # README에 적은 그대로 실행
7 docker compose up -d --build
8 docker compose ps -a
9
10 # 앱 로그와 DB 접속 확인
11 docker compose logs app
12 docker compose exec db psql -U engineer -d pipeline -c "SELECT
    current_database();"

```

확인 포인트: 볼륨까지 지운 상태에서 README의 명령만으로 환경이 다시 올라오면 검증 완료입니다.

week1 표는 볼륨을 지웠으니 없는 것이 정상입니다.

참고 pgAdmin에 등록해 둔 서버(week1-db)도 사라집니다.

admin 서비스에는 볼륨을 달지 않았기 때문에, 컨테이너를 새로 만들면 pgAdmin 설정도 처음 상태로 돌아갑니다.

로그인한 뒤 1.5H 5단계처럼 호스트 db로 다시 등록하면 됩니다.

실제 실행 화면 — down -v로 볼륨까지 지운 뒤 README의 명령만으로 다시 올렸습니다. ps -a · logs app · psql이 모두 정상입니다.

4 제출

- 제출 파일: docker-compose.yml · README.md (app 폴더의 Dockerfile도 함께 올리면 더 좋습니다)
- 사이트의 "산출물 제출" 페이지에서 제출합니다. GitHub 저장소 URL로 내도 됩니다.
- 제출 후 주차별 산출물 갤러리(패들릿)의 1주차 섹션에 [1주] 이름 — 산출물명 형식으로 올려 동료와 공유해 주세요.
- 4차시에서 썬 이미지 크기·재빌드 시간 수치도 함께 올리면 서로 비교하기 좋습니다.

1주차 총정리 · 산출물 제출 확인 (0.5H)

한 주 동안 쌓은 것을 한 장으로 되짚고, 2주차를 위해 환경을 정돈합니다.

1-5 · 정리·회고 0.5H · 1주차 총정리 · 산출물 제출 확인 슬라이드 11장

슬라이드: <https://raw.githubusercontent.com/aebonlee/materials/main/data-eng/decks/w1d5s4.pdf>

1주차에 지나온 길

차시	배운 것	남은 결과물
1-1	리눅스 명령과 파이프	~/data-eng/week1 · access.log · hello.sh
1-2	셸 스크립트와 종료 코드	greet.sh · check_file.sh · count_errors.sh · backup.sh · daily_report.sh
1-3	이미지·컨테이너·볼륨·포트	pgdata 볼륨 · 도커 명령 감각
1-4	Dockerfile과 레이어 캐시	app/ 폴더와 data-app 이미지
1-5	Compose로 환경 전체 정의	docker-compose.yml · README.md (산출물)

한 주를 관통한 생각은 하나입니다 — 손으로 하던 일을 파일로 옮겨 적어 두면 언제든지 똑같이 재현된다는 것.

명령을 스크립트로, 환경을 Dockerfile로, 환경 전체를 Compose로 옮겼습니다.

2주차부터는 이 위에 진짜 데이터를 엮습니다.

1주차가 이어지는 곳

이번 주에 익힌 리눅스·도커는 한 번 쓰고 끝나는 것이 아닙니다.

2~8주차 실습이 아래처럼 다시 꺼내 씁니다.

그 주차에서 막히면 해당 1주차 차시로 돌아와 보세요.

1주차에서 익힌 것	다시 나오는 곳
1-1 grep·tail·파이프	4·5·6·8주차 — docker compose logs 서비스명 grep ... 으로 컨테이너 로그에서 필요한 줄만 걸러 읽습니다
1-2 종료 코드·cron	5-5 — 마트 스크립트가 실패를 종료 코드로 알리고, 8주차 Airflow가 그 값을 보고 재시도를 정합니다. cron이 왜 부족한지는 8-1에서 다룹니다
1-3 docker run·-p·exec·이름 볼륨	2주차 — MySQL·MongoDB를 docker run -p로 띄우고 docker exec로 접속합니다. 4-1 — MinIO의 minio-data가 1-3과 같은 이름 볼륨입니다
1-4 이미지 층에 비밀값을 넣지 않는다	4-2 · 7-1 — API 키 같은 비밀값은 코드·이미지가 아니라 .env에 두고, .env는 Git에 올리지 않습니다
1-5 Compose·서비스 이름 규칙	4주차 MinIO·Iceberg(http://minio:9000) · 5주차 Spark(spark://spark-master:7077) · 6주차 Kafka(안은 kafka:29092, 밖은 localhost:9092) · 8주차 Airflow(공식 compose 파일)

오늘 개념만 본 healthcheck와 1-4에서 직접 써 본 Dockerfile 빌드는 이후 주차 실습에 다시 나오지 않습니다.

심화 과제로 익혀 두면 실무에서 바로 씁니다.

Compose 명령 요약

하려는 일	명령
문법 검사	<code>docker compose config</code>
기동 (빌드 포함)	<code>docker compose up -d --build</code>
상태 확인	<code>docker compose ps -a</code>
로그 보기	<code>docker compose logs 서비스명 (-f 실시간)</code>
안으로 들어가기	<code>docker compose exec 서비스명 bash</code> (bash가 없으면 <code>sh</code> — admin이 그렇습니다)
한 서비스만 재시작	<code>docker compose restart 서비스명</code>
정지·삭제 (볼륨 유지)	<code>docker compose down</code>
볼륨까지 삭제	<code>docker compose down -v</code> (되돌릴 수 없음)

2주차를 위한 정돈

bash

```

1  cd ~/data-eng/week1
2
3  # 이번 주 환경 내리기 (볼륨은 남겨 둡니다)
4  docker compose down
5
6  # 3차시의 pg 컨테이너가 남아 있으면 정리
7  docker rm pg 2>/dev/null
8
9  # 남은 것 확인
10 docker ps -a
11 docker images
12
13 # 2주차 폴더 미리 만들어 두기
14 mkdir -p ~/data-eng/week2

```

확인 포인트: 실행 중인 컨테이너가 없고, ~/data-eng 아래에 week1·week2 폴더가 있으면 정돈 완료입니다.

실제 실행 화면 — compose down이 week1의 컨테이너 셋과 네트워크를 지웁니다(볼륨 week1_pgdata는 남습니다). docker rm pg가 3차시의 pg를 지우고, ps -a에는 멈춰 있는(Exited) web2만 남았습니다. 실행 중인 컨테이너가 없으니 정돈 완료입니다. images에는 1주차에 받고 만든 이미지가 그대로 있습니다.

오늘의 체크리스트

- YAML의 들여쓰기 규칙(공백 2칸·탭 금지)을 설명할 수 있다.
- services·volumes·depends_on이 각각 무엇을 정의하는지 설명할 수 있다.
- 컨테이너 안에서 localhost와 서비스 이름의 차이를 실험으로 확인했다.
- docker compose up -d --build로 db·admin·app 3개 서비스를 한 번에 기동했다.
- pgAdmin에서 호스트 이름 db로 PostgreSQL에 연결했다.
- down 후 up을 해도 표가 남아 있는 것으로 볼륨의 데이터 보존을 확인했다.
- down -v 후 README의 명령만으로 환경이 재현되는지 스스로 검증했다.
- 주차 산출물(docker-compose.yml + README.md)을 제출했다.

자주 겪는 오류

주의 증상: yaml: line N: ... 파싱 오류 → 원인: YAML 들여쓰기가 어긋났거나 탭 문자가 섞였습니다.

→ 해결: 모든 들여쓰기를 공백 2칸으로 통일하고 탭을 제거합니다.

편집기의 "공백 표시" 기능을 켜면 찾기 쉽습니다.

docker compose config로 미리 검사할 수 있습니다.

주의 증상: port is already allocated (5432) → 원인: 3차시에 띄운 pg 컨테이너가 아직 5432 포트를 쓰고 있습니다.

→ 해결: docker stop pg로 정지한 뒤 docker compose up -d를 다시 실행합니다.

주의 증상: depends_on을 썼는데도 앱이 DB 연결에 실패한다 → **원인:** depends_on은 시작 순서만 보장할 뿐, DB가 접속 준비를 마쳤다는 보장이 아닙니다.

→ 해결: 잠시 후 재시도하면 연결됩니다.

근본 해결은 healthcheck와 depends_on의 condition 설정이며, 앱 쪽에 재시도 로직을 두는 것도 실무 표준입니다.

주의 증상: exec 줄을 여러 개 붙여 넣었는데 첫 줄 결과만 나오고 나머지는 아무 반응이 없다 → **원인:** docker compose exec 는 컨테이너의 입력(stdin)을 열어 둡니다.

한꺼번에 붙여 넣은 뒤 줄들이 터미널이 아니라 그 컨테이너의 입력으로 들어가 사라졌습니다.

→ 해결: exec 명령은 한 줄씩 붙여 넣고 결과를 본 뒤 다음 줄로 넘어갑니다.

결과가 안 나온 줄만 다시 치면 됩니다.

예상 질문

- Q. docker-compose와 docker compose는 다른가요? — 하이픈이 있는 것은 옛 파이썬 버전, 없는 것은 도커에 내장된 현재 버전입니다. 이번 과정은 전부 docker compose (공백)를 씁니다.
- Q. 파일 이름이 꼭 docker-compose.yml 이어야 하나요? — 기본값이 그것입니다. 다른 이름을 쓰려면 docker compose -f 파일명처럼 지정합니다.
- Q. 컨테이너가 계속 재시작을 반복해요 — docker compose logs 서비스명을 보세요. 대부분 환경변수 누락이나 설정 오류입니다.
- Q. 2주차부터는 무엇을 하나요? — 데이터베이스(MySQL·MongoDB)를 본격적으로 다룹니다. 2주차는 1-3에서 익힌 docker run·exec로 띄우고, 오늘 만든 Compose 형식은 4주차 MinIO부터 다시 씁니다(위 「1주차가 이어지는 곳」).

심화 과제 (선택)

- db 서비스에 healthcheck를 추가하고 depends_on을 condition: service_healthy 형태로 바꿔, app이 DB 준비 완료 후 시작되게 개선해 보세요.
- 비밀번호를 yml에 직접 쓰지 않고 .env 파일로 분리해 보세요. Compose는 같은 폴더의 .env를 자동으로 읽습니다.
- app 서비스에 volumes로 소스 폴더를 바인드 마운트해, 코드를 고치면 재빌드 없이 반영되게 만들어 보세요.

- 1주차 전체를 하나의 Git 저장소로 만들고 커밋해 보세요. .gitignore에 무엇을 넣어야 할지도 함께 생각해 보세요.

참고 링크

- Docker Compose 개요: <https://docs.docker.com/compose/>
- Compose 파일 레퍼런스: <https://docs.docker.com/reference/compose-file/>
- pgAdmin 컨테이너 배포 문서:
https://www.pgadmin.org/docs/pgadmin4/latest/container_deployment.html

이번 주 산출물

제출은 [산출물 제출](#) 페이지에서 하고, 제출 후 [주차별 산출물 갤러리](#)의 해당 주 섹션에 [1주] 이름 - 산출물명 형식으로 올려 동료와 공유해 주세요.

Built with the [LIVE method](#) by [이애본 \(Aebon Lee\)](#) · DreamIT Biz